

PROGRAMACIÓN IV

TP0 — Del Modelo Relacional al Backend

Normalización, Diagrama Entidad-Relación y Diseño de la Base de Datos como Fundamento del Proyecto Integrador

Trabajo Práctico Integrador y Preparatorio

Fase previa al desarrollo del backend en FastAPI (TP1)

ALUMNA

Villalba Mara Patricia

COMISIÓN

11

DOCENTE

Albornoz Lucas Ariel

FECHA DE ENTREGA

28/08/2026

Contenido

FASE 1 – INVESTIGACIÓN TEÓRICA Y JUSTIFICACIÓN	3
• Formal Normal	3
• Modelos en lo que conviene normalizar:.....	3
• Modelos en los que NO conviene normalizar:.....	3
Explicación de la Primera Forma Normal (1FN): atomicidad de los atributos, eliminación de grupos repetitivos y de atributos multivaluados:.....	3
Explicación de la Segunda Forma Normal (2FN): dependencia funcional completa respecto de la clave primaria, eliminación de dependencias parciales:.....	3
Explicación de la Tercera Forma Normal (3FN): eliminación de dependencias transitivas respecto de atributos no clave:	4
Descripción de las anomalías que la normalización previene, con al menos un ejemplo propio (no del dominio elegido) para cada una:.....	4
FASE 2 – Proceso de normalización (UNF -> 1FN -> 2FN -> 3FN)	5
Tabla original sin normalizar (UNF):.....	5
Tabla de aplicación de 1FN:.....	6
Tabla de aplicación de 2FN:.....	7
Tabla de aplicación de 3FN:.....	8
Lista final de las tablas en 3FN (con PK y FK).....	9
FASE 3 – Modelo del diagrama entidad - relación (der)	9
FASE 4 – Proyección al tp1: del modelo relacional a fastapi	10
Como cada entidad del DER se traducirá previsiblemente en un modelo SQLAlchemy (tabla del ORM):.....	10
Como se relacionan las claves foráneas y las tablas intermedias N:M con las relaciones (relationship) que deberán declararse entre modelos:.....	10
Qué entidades probablemente requerirán esquemas de Pydantic diferenciados para entrada (creación/edición) y salida (respuesta de la API), y por qué:.....	10
Una propuesta preliminar de qué recursos (entidades) tendrán sus propios endpoints REST en el TP1 (por ejemplo, mencionar que existirán rutas para gestionar la entidad principal y sus relaciones más relevantes), sin necesidad de detallar cada endpoint todavía:.....	10
Reflexión de cierre:.....	11

FASE 1 – INVESTIGACIÓN TEÓRICA Y JUSTIFICACIÓN

Definición de forma normal y por qué existe el proceso de normalización dentro del diseño de bases de datos relacionales:

- **Formal Normal** define básicamente lo que se entiende como esquema relacional bien formado, se consigue uno en base a la aplicación de varias formas normales, se dice que las relaciones están en una determinada forma normal si satisface un cierto conjunto de restricciones para cada forma normal. Se busca principalmente evitar duplicación de datos, inconsistencias y anomalías por modificaciones. El objetivo de la normalización es reducir la duplicación de datos y garantizar la integridad, a través de una serie de pasos (formas normales), descomponiendo una tabla grande y redundante en varias más pequeñas a través de relaciones lógicas.
 - Ventajas de la normalización: elimina redundancia, consistencia e integridad, mantenibilidad, facilidad de cumplir con reglas de negocio y escalabilidad.
 - Desventajas de la normalización: complejidad de diseño (requiere profundidad de análisis), cantidad de tablas (número elevado de tablas), impacto de rendimiento (muchas relaciones pueden ralentizará consultas), equilibrio entre normalizar y no hacerlo (difícil encontrar el balance).
- **Modelos en lo que conviene normalizar:** sistemas transaccionales como bancos o sistemas de pago, base de datos con datos regulados como historiales médicos o registros legales, entornos con almacenamiento limitado o costoso como bases móviles o sistemas embebidos.
- **Modelos en los que NO conviene normalizar:** sistema de reportes como power BI o sistemas de BI, sistemas de alto rendimiento de lectura como redes sociales o plataformas de anuncios, o casos donde los datos cambian poco, como por ejemplo un catálogo de productos históricos, o contenido estático básicamente.

Explicación de la Primera Forma Normal (1FN): atomicidad de los atributos, eliminación de grupos repetitivos y de atributos multivaluados:

- Una relación está en 1FN si todos sus atributos contienen un valor atómico, es decir, no hacen referencia a una lista de valores o grupos repetitivos, o sea, elimina grupos repetitivos, cada celda debe contener un único valor.
- Una tabla está en 1FN cuando:
 - Cada celda contiene un único valor atómico.
 - No hay repeticiones o listas dentro de una columna.
 - Cada fila representa una sola instancia de la entidad.

Explicación de la Segunda Forma Normal (2FN): dependencia funcional completa respecto de la clave primaria, eliminación de dependencias parciales:

- Una relación está en 2FN si está en 1FN y además cada uno de los atributos no claves son dependientes funcionales completos de cada clave candidata. Es decir, eliminar dependencias parciales de claves compuestas.

Explicación de la Tercera Forma Normal (3FN): eliminación de dependencias transitivas respecto de atributos no clave:

- Una relación está en 3FN si y solo si está en 2FN y todos los atributos claves dependen de manera no transitiva de la clave primaria, de manera más simple, eliminar dependencias transitivas.

Descripción de las anomalías que la normalización previene, con al menos un ejemplo propio (no del dominio elegido) para cada una:

- Anomalías de inserción: ocurre cuando no se puede ingresar una ocurrencia hasta que se tenga un hecho adicional cerca de otra entidad.
- Anomalías de modificación: ocurre cuando se modifica un valor, y no se verifican los valores ingresados, los que finalmente son consolidados en la base de datos, una solución a este problema posible es dividir la relación en dos relaciones.
- Anomalías de borrado: ocurre cuando se elimina una tupla o un valor específico y este hecho afecta a otras entidades.
- Dicho esto, imaginemos un sistema bancario (como en el ejemplo de ventajas de normalización) donde tenemos una tabla única que lleve `id_cuenta`, `titular_cuenta`, `id_transaccion`, `monto` y `sucursal_cuenta`:
 - Anomalía de inserción: no podemos dar de alta una cuenta nueva sin que tenga al menos una transacción cargada, porque la fila “no existe” sin un `id_transaccion`, un cliente que recién abre la cuenta y todavía no opero, no podría estar en el sistema.
 - Anomalía de borrado: si borramos la única transacción de una cuenta, por ejemplo, se revierte por error, se borra con ella el titular y la sucursal de esa cuenta, perdemos la cuenta entera de nuestro sistema, aunque siga existiendo en la realidad.
 - Anomalía de modificación: si la cuenta cambia de sucursal y esa cuenta tiene 200 transacciones, es decir, 200 filas, hay que actualizar la sucursal en las 200 filas. Si falla una, nos queda una transacción vieja apuntando a la sucursal anterior, es una inconsistencia grave en un sistema bancario.
- O también tomemos el ejemplo de los historiales médicos, diciendo que tenemos una tabla donde hay `id_paciente`, `nombre_paciente`, `diagnostico`, `medico_tratante`:
 - Anomalía de inserción: no podemos registrar un paciente nuevo en el sistema hasta que tenga al menos una consulta cargada.
 - Anomalía de borrado: si se elimina por error la única consulta registrada de un paciente, se pierde también su ficha, no solo esa consulta puntual.
 - Anomalía de modificación: si cambia el medico tratante de un paciente crónico con 50 consultas históricas, hay que actualizar las 50 filas, un olvido genera un historial medico con datos contradictorios sobre quien lo atiende, algo delicado en un dominio regulado.
- Reflexión final sobre por qué resulta imprescindible normalizar el modelo antes de “llevarlo a código”, es decir, antes de traducirlo a tablas físicas, modelos ORM o endpoints de una API.
 - Tenemos varias razones de porque es imprescindible normalizar el modelo antes de pasar al código, por ejemplo; primero el costo el corregir crece con el tiempo, como fue mencionado en clases, corregir un modelo de diagrama cuesta

minutos o incluso segundos, es borrar o mover una flecha y listo, mientras que corregirlo después de tener código funcionando y datos reales cargados implica migraciones de base de datos, reescribir endpoints que ya consumían esa estructura y en el peor caso perder o corromper datos ya existentes. Otro ejemplo también es arrastrar un modelo mal normalizado al código, esas anomalías de inserción, borrado o modificación que se vieron en los ejemplos anteriores van a aparecer como bugs difíciles de rastrear, porque no son errores de código en sí, sino errores de diseño escondidos detrás del código. También es algo que vi en la practica cuando hice la tabla de UNF sin normalizar que generaba redundancia real con nombres de clientes repetidos o categorías repetidas y como aplica 1FN -> 2FN -> 3FN, esos problemas desaparecieron.

FASE 2 – Proceso de normalización (UNF -> 1FN -> 2FN -> 3FN)

Tabla original sin normalizar (UNF):

id_venta	fecha_venta	id_cliente	nombre_cliente	telefono_cliente	email_cliente	direccion_cliente	id_producto	nombre_producto	talle	color	id_categoria
1	15/08/2026	1	Ana Gómez	381-4551122, 381-6789990	ana.gomez@mail.com	Av. Roca 1234, San Miguel de Tucumán, CP 4000	1	Vestido floreado	M	Rojo	1
1	15/08/2026	1	Ana Gómez	381-4551122, 381-6789990	ana.gomez@mail.com	Av. Roca 1234, San Miguel de Tucumán, CP 4000	2	Cinto de cuero	Único	Negro	2
2	16/08/2026	2	Marta Ruiz	381-5556677	marta.ruiz@mail.com	Belgrano 890, San Miguel de Tucumán, CP 4000	1	Vestido floreado	S	Azul	1
3	17/08/2026	3	Carla Díaz	381-4443322, 381-7778899, 381-9990011	carla.diaz@mail.com , cdiaz.trabajo@mail.com	Mitre 2200, San Miguel de Tucumán, CP 4001	3	Campera de jean	L	Celeste	3
3	17/08/2026	3	Carla Díaz	381-4443322, 381-7778899, 381-9990011	carla.diaz@mail.com , cdiaz.trabajo@mail.com	Mitre 2200, San Miguel de Tucumán, CP 4001	4	Remera básica	M	Blanco	4

id_categoria	nombre_categoria	precio_unitario	cantidad	medio_de_pago
1	Vestidos	\$15.000	2	Efectivo, Tarjeta débito
2	Accesorios	\$15.000	1	Efectivo, Tarjeta débito
1	Vestidos	\$15.000	1	Efectivo
3	Abrigos	\$22.000	1	Tarjeta crédito
4	Remeras	\$7.500	2	Tarjeta crédito

Tabla de aplicación de 1FN:

id_venta	fecha_venta	id_cliente	nombre_cliente	calle	ciudad	codigo_postal
1	15/08/2026	1	Ana Gómez	Av. Roca 1234	San Miguel de Tucumán	4000
1	15/08/2026	1	Ana Gómez	Av. Roca 1234	San Miguel de Tucumán	4000
2	16/08/2026	2	Marta Ruiz	Belgrano 890	San Miguel de Tucumán	4000
3	17/08/2026	3	Carla Díaz	Mitre 2200	San Miguel de Tucumán	4001
3	17/08/2026	3	Carla Díaz	Mitre 2200	San Miguel de Tucumán	4001

id_producto	nombre_producto	talle	color	id_categoria	nombre_categoria
1	Vestido floreado	M	Rojo	1	Vestidos
2	Cinto de cuero	Único	Negro	2	Accesorios
1	Vestido floreado	S	Azul	1	Vestidos
3	Campera de jean	L	Celeste	3	Abrigos
4	Remera básica	M	Blanco	4	Remeras

precio_unitario	cantidad
\$15.000	2
\$15.000	1
\$15.000	1
\$22.000	1
\$7.500	2

telefono_cliente

id_telefono	id_cliente	telefono
1	1	381-4551122
2	1	381-6789990
3	2	381-5556677
4	3	381-4443322
5	3	381-7778899
6	3	381-9990011

email_cliente

id_email	id_cliente	email
1	1	ana.gomez@mail.com
2	2	marta.ruiz@mail.com
3	3	carla.diaz@mail.com
4	3	cdiaz.trabajo@mail.com

medio_pago_venta

id_medio_pago	id_venta	medio_pago
1	1	Efectivo
2	1	Tarjeta débito
3	2	Efectivo
4	3	Tarjeta crédito

Para llegar a 1FN se resolvieron los dos problemas de atomicidad detectados en la tabla UNF:

- Los campos telefono_cliente, email_cliente y medio_de_pago no eran atómicos, ya que contenían varios valores separados por coma en una sola celda (por ejemplo, un cliente con dos teléfonos). Se extrajeron a tablas propias (TELEFONO_CLIENTE, EMAIL_CLIENTE, MEDIO_PAGO_VENTA), vinculadas mediante id_cliente o id_venta según corresponda, de forma que cada fila contenga un único valor. El campo direccion_cliente tampoco era atómico, ya que combinaba calle, ciudad y código postal en un solo texto. Al ser un único valor por cliente (no una lista), no requirió una tabla aparte: se descompuso directamente en tres columnas separadas (calle, ciudad, codigo_postal) dentro de la misma tabla principal. Con estos cambios, cada campo de la tabla contiene un único valor atómico y no quedan grupos repetitivos, cumpliendo así el requisito de 1FN.

Tabla de aplicación de 2FN:

VENTA

id_venta	fecha_venta	id_cliente
1	15/08/2026	1
2	16/08/2026	2
3	17/08/2026	3

CLIENTE

id_cliente	nombre_cliente	calle	ciudad	codigo_postal
1	Ana Gómez	Av. Roca 1234	San Miguel de Tucumán	4000
2	Marta Ruiz	Belgrano 890	San Miguel de Tucumán	4000
3	Carla Díaz	Mitre 2200	San Miguel de Tucumán	4001

TELEFONO_CLIENTE

id_telefono	id_cliente	telefono
1	1	381-4551122
2	1	381-6789990
3	2	381-5556677
4	3	381-4443322
5	3	381-7778899
6	3	381-9990011

EMAIL_CLIENTE

id_email	id_cliente	email
1	1	ana.gomez@mail.com ↗
2	2	marta.ruiz@mail.com ↗
3	3	carla.diaz@mail.com ↗
4	3	cdiaz.trabajo@mail.com ↗

PRODUCTO

id_producto	nombre_producto	id_categoria	nombre_categoria
1	Vestido floreado	1	Vestidos
2	Cinto de cuero	2	Accesorios
3	Campera de jean	3	Abrigos
4	Remera básica	4	Remeras

DETALLE_VENTA

id_detalle	id_venta	id_producto	talle	color	cantidad	precio_unitario
1	1	1	M	Rojo	2	\$15.000
2	1	2	Único	Negro	1	\$15.000
3	2	1	S	Azul	1	\$15.000
4	3	3	L	Celeste	1	\$22.000
5	3	4	M	Blanco	2	\$7.500

MEDIO_PAGO_VENTA

id_medio_pago	id_venta	medio_pago
1	1	Efectivo
2	1	Tarjeta débito
3	2	Efectivo
4	3	Tarjeta crédito

Con la clave compuesta (id_venta + id_producto) establecida, analizamos qué atributos dependen de la clave completa y cuáles dependen solo de una parte de ella (dependencia parcial):

- fecha_venta depende únicamente de id_venta, no de id_producto: la fecha de una venta no cambia según qué producto se haya vendido en ella. Por eso se separó en la tabla VENTA junto con id_cliente.
- nombre_cliente, calle, ciudad y codigo_postal dependen únicamente de id_cliente, que ni siquiera es parte de la clave primaria de la tabla original. Por eso se separaron en la tabla CLIENTE.
- nombre_producto, id_categoria y nombre_categoria dependen únicamente de id_producto, no de id_venta: son características propias del producto, independientes de en qué venta aparezca. Por eso se separaron en la tabla PRODUCTO.
- talle, color, cantidad y precio_unitario sí dependen de la combinación completa (id_venta + id_producto), ya que representan el detalle específico de qué talle/color/cantidad/precio tuvo ese producto en esa venta puntual. Por eso

permanecen en la tabla DETALLE_VENTA, ahora identificada con clave surrogada id_detalle (según lo justificado previamente).

- Las tablas TELEFONO_CLIENTE, EMAIL_CLIENTE y MEDIO_PAGO_VENTA, extraídas ya en 1FN, cumplen 2FN sin cambios adicionales: sus atributos (telefono, email, medio_pago) dependen de la clave completa de cada una de esas tablas (id_telefono, id_email, id_medio_pago respectivamente).

Con esta separación, ningún atributo no clave depende de una parte de la clave primaria original: se eliminaron todas las dependencias parciales, cumpliendo así el requisito de 2FN.

Tabla de aplicación de 3FN:

tabla	¿hay algún otro atributo no clave que dependa de otro atributo no clave?
VENTA	no, fecha_venta depende de id_venta
CLIENTE	no, todos los atributos como nombre, calle, ciudad, codigo_postal, dependen directamente de id_cliente
DETALLE_VENTA	No, talle, color y cantidad dependen de id_detalle. No usamos (id_venta + id_producto) como clave candidata porque no garantiza unicidad: un mismo producto podría venderse dos veces dentro de la misma venta en talles o colores distintos. Por eso se optó por un id surrogado.
PRODUCTO	sí, id_producto -> nombre_categoria, pero en realidad nombre_categoria no depende del producto en sí, sino que depende de a que categoría pertenece ese producto, es decir, id_producto -> categoria del producto -> nombre_categoria, esa sería la dependencia transitiva clásica, un atributo no clave como categoria, determinando a otro atributo no clave como nombre_categoria

- Dicho esto, como encontramos una dependencia transitiva, tenemos que sacar el atributo dependiente hacia una tabla nueva:

CATEGORÍA

id_categoria	nombre_categoria
1	Vestidos
2	Accesorios
3	Abrigos
4	Remeras

- Ahora la tabla "producto" quedaría de la siguiente manera:
 - Se separó categoría como tabla propia para eliminar la redundancia de repetir el nombre de categoría en cada producto.

id_producto	nombre_producto	id_categoria
1	Vestido floreado	1
2	Cinto de cuero	2
3	Campera de jean	3
4	Remera básica	4

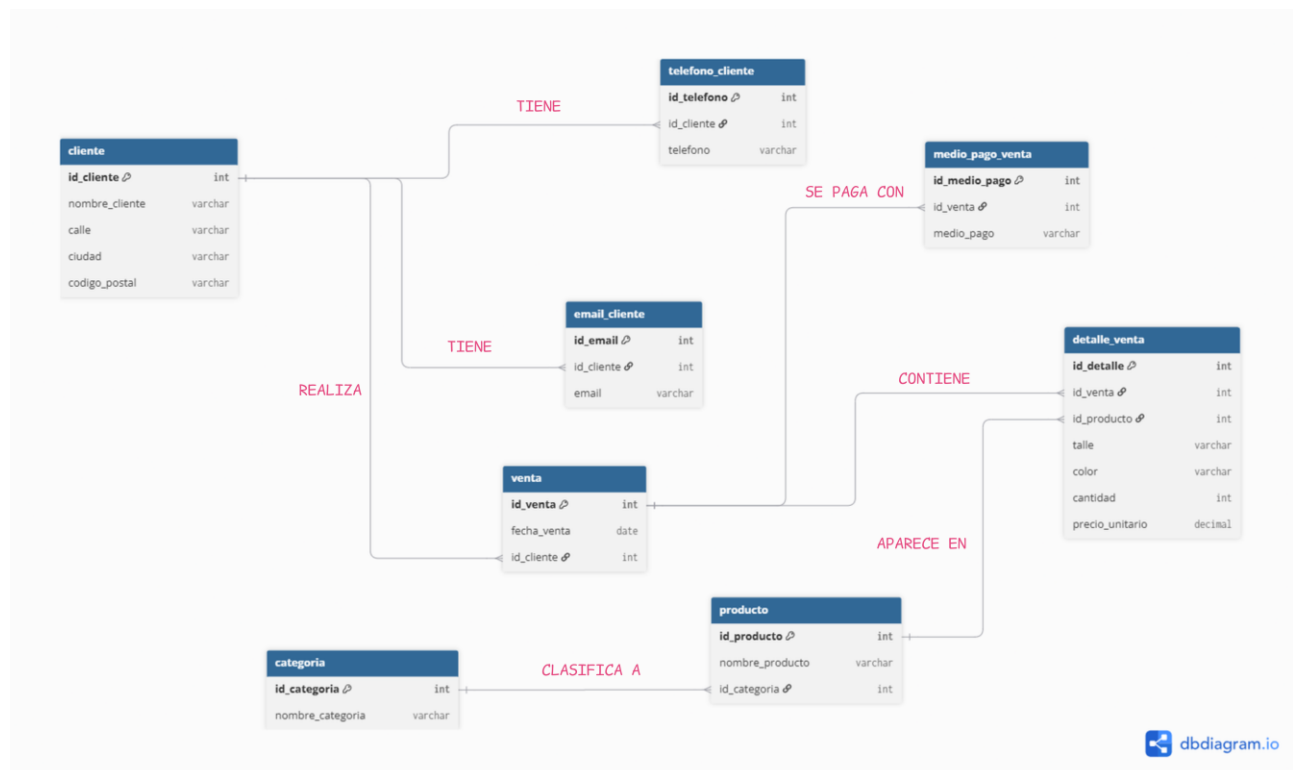
Aunque precio_unitario en PRODUCTO no viola ninguna forma normal, genera un problema práctico: si el precio del producto se actualiza en el futuro, los subtotales de ventas ya realizadas se recalcularían con el precio nuevo, perdiendo el precio real al que se vendió. Para evitar esto, precio_unitario se guarda en DETALLE_VENTA (el precio vigente en el momento de esa venta), y no en PRODUCTO. PRODUCTO puede conservar un precio de catálogo/actual si se necesita para nuevas ventas, pero el histórico de ventas necesita su propio precio "congelado".

Lista final de las tablas en 3FN (con PK y FK)

1. **CLIENTE:** id_cliente (PK), nombre_cliente, calle, ciudad, codigo_postal
2. **VENTA:** id_venta (PK), fecha_venta, id_cliente (FK)
3. **CATEGORIA:** id_categoria (PK), nombre_categoria
4. **PRODUCTO:** id_producto (PK), nombre_producto, id_categoria (FK)
5. **DETALLE_VENTA:** id_detalle (PK), id_venta (FK), id_producto (FK), talla, color, cantidad, precio_unitario
6. **TELEFONO_CLIENTE:** id_telefono (PK), id_cliente (FK), telefono
7. **EMAIL_CLIENTE:** id_email (PK), id_cliente (FK), email
8. **MEDIO_PAGO_VENTA:** id_medio_pago (PK), id_venta (FK), medio_pago

FASE 3 – Modelo del diagrama entidad - relación (der)

- El diagrama fue realizado utilizando notación Crow's Foot (pata de cuervo), a través de la herramienta dbdiagram.io.



FASE 4 – Proyección al tp1: del modelo relacional a fastapi

Como cada entidad del DER se traducirá previsiblemente en un modelo SQLAlchemy (tabla del ORM):

- Cada una de las 8 entidades del DER se va a traducir en una clase Python que hereda de la clase base de SQLAlchemy, donde el nombre de la clase representa la tabla. Cada atributo de la entidad se va a traducir en un atributo de esa clase, indicando su tipo de dato. La clave primaria de cada entidad se marcará como tal dentro del modelo, y las claves foráneas se declararán indicando a qué tabla y columna hacen referencia, estableciendo así el vínculo entre entidades a nivel de base de datos. Como ya contamos con PK y FK bien definidas desde el DER, este mapeo resulta prácticamente directo: es una de las ventajas concretas de haber normalizado antes de programar.

Como se relacionan las claves foráneas y las tablas intermedias N:M con las relaciones (relationship) que deberán declararse entre modelos:

- Cada FK simple (por ejemplo, id_categoria en PRODUCTO) se va a traducir en una relación 1:N entre CATEGORIA y PRODUCTO mediante relationship(). El caso particular es DETALLE_VENTA, que resuelve la relación N:M entre VENTA y PRODUCTO: al tener dos FK (id_venta e id_producto), desde VENTA se podrá navegar la lista de sus detalles, y desde PRODUCTO también. Como DETALLE_VENTA tiene atributos propios (talle, color, cantidad, precio_unitario), se modela como una entidad completa con sus propias relaciones, y no como una tabla puente "invisible" de solo dos columnas.

Qué entidades probablemente requerirán esquemas de Pydantic diferenciados para entrada (creación/edición) y salida (respuesta de la API), y por qué:

- VENTA y DETALLE_VENTA necesitan schemas separados para entrada y salida: en la creación no se envía id_venta ni id_detalle (los genera la base de datos), pero sí se devuelven en la respuesta. Además, en la salida de DETALLE_VENTA probablemente convenga devolver los datos del producto anidados (nombre, categoría), no solo su id, algo que en la entrada no aplica. CLIENTE también podría requerir un schema de salida que incluya sus teléfonos y emails anidados, distinto del schema simple usado para crearlo o editarlo.

Una propuesta preliminar de qué recursos (entidades) tendrán sus propios endpoints REST en el TP1 (por ejemplo, mencionar que existirán rutas para gestionar la entidad principal y sus relaciones más relevantes), sin necesidad de detallar cada endpoint todavía:

- Los recursos principales del negocio (CLIENTE, PRODUCTO, CATEGORIA y VENTA) tendrían sus propios endpoints REST (/clientes, /productos, /ventas, etc.) con las operaciones habituales de creación, lectura, actualización y borrado. DETALLE_VENTA, en cambio, no tendría sentido como endpoint independiente, ya que no existe sin una venta asociada: se gestionaría anidado, por ejemplo, bajo /ventas/{id}/detalles. El mismo criterio aplicaría para

TELEFONO_CLIENTE, EMAIL_CLIENTE y MEDIO_PAGO_VENTA, gestionados como sub-recursos de CLIENTE y VENTA respectivamente.

Reflexión de cierre:

A esta altura del TP0 quedó bastante claro que normalizar no es algo teórico antes de "lo importante": es evitar que lo importante se rompa después. El caso del precio_unitario que movimos de PRODUCTO a DETALLE_VENTA es un buen ejemplo de eso, no fue un error de normalización en sí, pero si esa decisión se tomaría recién en el TP1, con el modelo de SQLAlchemy ya escrito y quizás con datos de prueba ya cargados, corregirla implica tocar el modelo, el schema de Pydantic que lo expone y el endpoint que ya lo estaba devolviendo, todo por un problema que se podría haber visto con solo mirar el diagrama a tiempo. Ahí está la diferencia real entre corregir en un diagrama y corregir en código: en el diagrama es borrar una línea, en el código es una tarde entera de refactor y probar que no rompiste nada más. Y no es solo tiempo perdido: un modelo mal pensado arrastra directo las anomalías que vimos en la Fase 1 (inserción, borrado, modificación) al sistema real, como bugs de datos que después son un dolor de cabeza rastrear porque el error no está en el código, está en cómo se pensaron las tablas. Por eso este TP0, aunque parezcan "solo diagramas", es literalmente la base de lo que viene, cualquier cosa que dejemos mal pensada acá se va a notar cuando empecemos a escribir el TP1.